

DL Midsem — Full Question Bank

PYQs, Additional Questions, Two Mock Papers, and Complete Answer Key

Some PYQs are worded near-identically across multiple papers with different mark weights (e.g. the perceptron convergence proof appears 3 times). These are merged into one entry below, listing every paper/mark combination it appeared under, so you see the pattern instead of reading the same question three times.

Contents

1	Part A — PYQ Bank	2
2	Part B — Additional Practice Questions	4
3	Part C — Mock Paper 1	(35 marks) 5
4	Part D — Mock Paper 2	(37 marks) 6
5	Part E — Answer Key	7
5.1	PYQ Bank Answers	7
5.2	Additional Questions Answers	11
5.3	Mock Paper 1 Answers	13
5.4	Mock Paper 2 Answers	13

1 Part A — PYQ Bank

PYQ1 [3]. Explain the McCulloch–Pitts (MP) neuron along with its limitations. How does the perceptron model overcome these limitations? *(CS6510-1 Q1A)*

PYQ2 [3/4/4]. If the sets P and N are finite and linearly separable, prove that the perceptron learning algorithm updates the weight vector w_t a finite number of times such that the two sets are separated. *(CS6510-1 Q1B / CS6510-2 Q6 / CS6510-9 Q3b)*

PYQ3 [3]. Determine whether it is possible to implement the XOR gate logic using a perceptron. *(CS6510-1 Q2A)*

PYQ4 [3]. Explain three regularization methods to overcome the issue of overfitting of a complex model. *(CS6510-1 Q3)*

PYQ5 [2]. A 1-hidden-layer neural network with k neurons, n inputs and m outputs with bias at each neuron will have how many total parameters? *(CS6510-2 Q1)*

PYQ6 [2]. A nonlinear function passes 4 if its input is non-negative, else -2 . A friend recommends using this non-linearity with the Adam optimizer. Would you follow this advice? Why or why not? *(CS6510-2 Q3)*

PYQ7 [2]. Sketch the early-stopping diagram: what goes on each axis, what do the two curves represent, and where do you stop? *(CS6510-2 Q4)*

PYQ8 [4]. Prove that moving in the direction opposite to the gradient minimizes the function. Explain the backpropagation algorithm using the gradient descent update rule. *(CS6510-2 Q7)*

PYQ9 [5/4]. What problem occurs if the learning rate is too high or too low? Why is an adaptive learning rate w.r.t. the cumulative sum of gradients used in AdaGrad? Explain how RMSProp overcomes AdaGrad's problem, with weight update equations. *(CS6510-2 Q8 / CS6510-5 Q1c)*

PYQ10 [4]. A network with sigmoid activation at every neuron has given weights, inputs and target output. Compute the value of W_4, W_7, W_8 after 1 epoch of backpropagation using SGD with $\eta = 0.1$. *(CS6510-2 Q9)*

PYQ11 [4]. What is the softmax activation function? How is it different from sigmoid, and when would softmax be used instead of sigmoid? *(CS6510-7 Q1b)*

PYQ12 [10]. Explain the Adam optimization algorithm. What are its key components, and how does it differ from standard SGD? What is the advantage of using momentum (first moment estimate)? *(CS6510-7 Q2a)*

PYQ13 [6]. Discuss overfitting and underfitting. Explain L1 and L2 regularization. *(CS6510-7 Q2b)*

PYQ14 [5/5]. Given a filter of width F and depth d , applied to an image of dimension (W_1, H_1) , derive (W_2, H_2) assuming S strides and P padding. If there are K kernels, what is the number of parameters formed? *(CS6510-3 Q1a / CS6510-4 Q1A)*

PYQ15 [5/5]. Using backpropagation for a given neural network (sigmoid activation), compute the output of nodes y_3, y_4, y_5 . *(CS6510-3 Q1b / CS6510-4 Q1B)*

PYQ16 [5/5/5]. Explain the bias-variance tradeoff with an appropriate diagram. Derive the equation for predicted error in terms of bias and variance. *(CS6510-3 Q3a / CS6510-4 Q3 / CS6510-6 Q3a)*

PYQ17 [5]. An $n \times n$ image I is convolved once with a 1×1 filter and once with an $n \times n$ filter (stride 1, no padding). What are the conclusions in each case, and why? *(CS6510-3 Q3b)*

- PYQ18 [5].** Derive the generic term to calculate $\text{grad}(w_i)$ for a given small ANN. Find the expressions for $\text{grad}(w_1)$, $\text{grad}(w_2)$, $\text{grad}(w_3)$. *(CS6510-3 Q4a)*
- PYQ19 [5].** Differentiate between: (i) Stochastic GD (ii) Momentum GD (iii) Nesterov Accelerated GD. *(CS6510-4 Q2A)*
- PYQ20 [2].** Design a McCulloch–Pitts neuron to model a 3-bit logical AND. *(CS6510-5 Q1b)*
- PYQ21 [4].** True/False, correct if false: (a) Simple models have high bias and high variance. (b) High bias and low variance are desirable. (c) Variance contributes to error while bias does not. (d) In NAG, the gradient term is added to the old weight first, then the momentum term. *(CS6510-5 Q2a)*
- PYQ22 [3].** Show why Adam’s bias correction naturally disappears as the number of steps gets large. *(CS6510-5 Q3b)*
- PYQ23 [4].** Why is an activation function used in a neural network? What are the characteristics of a good one? Explain sigmoid’s merits/drawbacks via its derivative. *(CS6510-5 Q4a)*
- PYQ24 [6].** Binary classifier (sweet watermelon = 1). (i) One reason MSE is a poor loss here. (ii) BCE formula for one example. (iii) Loss for $\hat{y} = 0.9$ on a negative example. (iv) Average BCE cost J over 3 examples: $Y = (1, 0, 0)$, $\hat{Y} = (0.1, 0.2, 0.7)$. *(CS6510-5 Q4b)*
- PYQ25 [5].** For a given CNN classifier (layer sequence INPUT $128 \times 128 \times 3$, CONV-9-32, POOL-2, CONV-5-64, POOL-2, CONV-5-64, POOL-2, FC-10, FC-3), calculate parameters and feature-map dimensions at each layer. *(CS6510-5 Q5a)*
- PYQ26 [2].** What is the minimum number of perceptron units sufficient to model 3-variable XOR with a single hidden layer? Draw the model. *(CS6510-5 Q5b)*
- PYQ27 [5].** A given input image is convolved (stride 1) with a given filter. Find the output. Apply 2×2 max pooling, stride 1. What padding keeps the convolution output the same size as the input? *(CS6510-5 Q6a)*
- PYQ28 [2].** Mention at least 8 forms of regularization used in deep learning. *(CS6510-5 Q6c)*
- PYQ29 [2].** Which loss function is used for a classification problem? Define it mathematically. *(CS6510-9 Q1a)*

2 Part B — Additional Practice Questions

- O1 [2].** Design a McCulloch–Pitts neuron to model a 2-input NOR gate.
- O2 [3].** If a perceptron’s training data has margin $\gamma = 0.5$ and $\max_i \|x_i\| = R = 5$, what is the maximum number of mistakes the perceptron algorithm can make before converging?
- O3 [5].** A single scalar parameter $w_0 = 2.0$ is trained with Adam ($\eta = 0.01, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$). Gradients observed: $g_1 = 1.2, g_2 = -0.5$. Compute w_1 and w_2 .
- O4 [4].** At $w_0 = 1.0$ with existing velocity $v_{\text{prev}} = 0.3, \beta = 0.9, \eta = 0.1$, for $f(w) = w^2$: compute the next weight w_1 using (a) plain Momentum, (b) NAG. Compare the two updates and explain why they differ.
- O5 [5].** For a linear layer $y = \sum_{i=1}^n w_i x_i$ with w_i, x_i independent, zero-mean, derive the variance $\text{Var}(w)$ that keeps $\text{Var}(y) = \text{Var}(x)$ (Xavier-style forward-variance argument).
- O6 [3].** Explain, with reasoning (not just the formula), why He initialization uses $\text{Var}(w) = 2/n_{in}$ for ReLU networks instead of Xavier’s $1/n_{in}$.
- O7 [5].** For inverted dropout with keep-probability p (output scaled by $1/p$ during training), prove that $\mathbb{E}[\text{output}] = x$ (the un-dropped value), and explain why this means no rescaling is needed at test time.
- O8 [4].** For batch normalization $z_i = \gamma \hat{x}_i + \beta$ over a mini-batch of size m , given the upstream gradient $\delta_i = \partial L / \partial z_i$ for each i , derive $\partial L / \partial \gamma$ and $\partial L / \partial \beta$.
- O9 [5].** A CNN has: Layer 1 – input $32 \times 32 \times 3$, CONV $F=5, K=16, S=1, P=2$; Layer 2 – CONV $F=4, K=32, S=2, P=0$. Compute the output dimensions and parameter count of each layer.
- O10 [5].** A network has $x_1 = 0.5, x_2 = -0.2$ feeding a single **tanh** hidden neuron h (weights $w_1 = 0.6, w_2 = 0.3$, bias $b_h = 0.1$), which feeds a sigmoid output $\hat{y}$ (weight $w_3 = 0.8$, bias $b_o = -0.2$). Target $y = 0.4, \eta = 0.15$, loss is MSE. Compute updated w_1, w_3 after one SGD step.

3 Part C — Mock Paper 1

(35 marks)

M1.1 [3]. Explain the McCulloch–Pitts neuron and its limitations. How does the perceptron overcome them?

M1.2 [4]. Prove the perceptron convergence theorem for finite, linearly separable sets P, N .

M1.3 [4]. A network (2 inputs, 2 sigmoid hidden neurons, 1 sigmoid output) has $x_1 = 0.4, x_2 = 0.3$; weights $W_1(x_1 \rightarrow h_1) = 0.25$, $W_2(x_2 \rightarrow h_1) = 0.35$, $W_3(b_1) = 0.1$, $W_4(x_1 \rightarrow h_2) = 0.4$, $W_5(x_2 \rightarrow h_2) = 0.2$, $W_6(b_2) = 0.15$, $W_7(b_o) = 0.1$; target $y = 0.25$, $\eta = 0.2$. Compute updated W_2, W_6, W_7 after one SGD step.

M1.4 [5]. Differentiate SGD, Momentum GD, and NAG.

M1.5 [5]. Explain AdaGrad and RMSProp with their weight-update equations, and how RMSProp fixes AdaGrad's core problem.

M1.6 [5]. Derive (W_2, H_2) and the parameter count for a convolution layer given filter width F , depth d , K kernels, stride S , padding P .

M1.7 [6]. A spam classifier gives predictions $\hat{Y} = (0.7, 0.4, 0.3, 0.6)$ for true labels $Y = (1, 1, 0, 0)$. (i) Why is MSE a poor loss choice here? (ii) Write the BCE formula. (iii) Compute the loss for $y = 1, \hat{y} = 0.8$. (iv) Compute the average BCE cost J over the 4 given examples.

M1.8 [3]. Show why Adam's bias correction disappears as $t \rightarrow \infty$.

4 Part D — Mock Paper 2

(37 marks)

M2.1 [3]. Prove that XOR cannot be implemented by a single perceptron.

M2.2 [5]. Explain the bias-variance tradeoff with a diagram, and derive predicted error = Bias² + Variance + Noise.

M2.3 [6]. Discuss overfitting and underfitting. Explain L1 and L2 regularization.

M2.4 [4]. What is softmax? How does it differ from sigmoid, and when should each be used?

M2.5 [10]. Explain the Adam optimizer in detail: its components, how it differs from SGD, and the advantage of momentum.

M2.6 [5]. A 5×5 input $\begin{pmatrix} 2 & 0 & 1 & 3 & 1 \\ 1 & 2 & 0 & 1 & 2 \\ 0 & 1 & 3 & 0 & 1 \\ 2 & 1 & 0 & 2 & 1 \\ 1 & 0 & 2 & 1 & 3 \end{pmatrix}$ is convolved (stride 1, no padding) with the filter

$\begin{pmatrix} 0 & 1 & 0 \\ 1 & -4 & 1 \\ 0 & 1 & 0 \end{pmatrix}$. Find the output, apply 2×2 max pooling (stride 1), and state the padding needed to keep the convolution output the same size as the input.

M2.7 [2]. Mention at least 8 forms of regularization used in deep learning.

M2.8 [2]. A 1-hidden-layer network has $n = 6$ inputs, $k = 8$ hidden neurons, $m = 4$ outputs, bias at every neuron. Find the total parameter count.

5 Part E — Answer Key

5.1 PYQ Bank Answers

A-PYQ1. MP neuron: binary inputs; excitatory inputs (weight 1) summed and compared to a threshold θ ; any active inhibitory input forces output 0 regardless. $y = 1$ iff $\sum x_i \geq \theta$ and no inhibitory input is active. **Limitations:** no learning (weights/threshold fixed by hand), only Boolean inputs, only linearly separable functions, all excitatory weights forced equal. **Perceptron overcomes this** by using learnable, real-valued weights and bias updated via $w \leftarrow w + \eta(y - \hat{y})x$, allowing real-valued inputs and differently-weighted inputs — though it remains a linear threshold device (still can't solve XOR).

A-PYQ2 (full proof). Let w^* ($\|w^*\| = 1$) separate the data with margin γ : $y_i(w^* \cdot x_i) \geq \gamma \forall i$. Let $R = \max_i \|x_i\|$. Let w_k be the weight after k mistake-driven updates, $w_0 = 0$; on the k -th mistake (x, y) , $y(w_{k-1} \cdot x) \leq 0$ and $w_k = w_{k-1} + yx$.

Lower bound: $w_k \cdot w^* = w_{k-1} \cdot w^* + y(x \cdot w^*) \geq w_{k-1} \cdot w^* + \gamma \Rightarrow w_k \cdot w^* \geq k\gamma$ (induction).

Upper bound: $\|w_k\|^2 = \|w_{k-1}\|^2 + 2y(w_{k-1} \cdot x) + \|x\|^2 \leq \|w_{k-1}\|^2 + R^2 \Rightarrow \|w_k\|^2 \leq kR^2$ (induction, using $y(w_{k-1} \cdot x) \leq 0$).

Combine (Cauchy-Schwarz): $k\gamma \leq w_k \cdot w^* \leq \|w_k\| \leq \sqrt{k}R \Rightarrow k \leq R^2/\gamma^2$.

The number of mistakes is bounded by the fixed constant R^2/γ^2 , so the algorithm must terminate (converge) after finitely many updates. ■

A-PYQ3. Not possible. Suppose w_1, w_2, b realize XOR via $\hat{y} = 1 \iff w_1x_1 + w_2x_2 + b \geq 0$. Then: $(0, 0) \rightarrow 0 : b < 0$; $(1, 1) \rightarrow 0 : w_1 + w_2 + b < 0$; $(0, 1) \rightarrow 1 : w_2 + b \geq 0$; $(1, 0) \rightarrow 1 : w_1 + b \geq 0$. Adding the last two: $w_1 + w_2 + 2b \geq 0 \Rightarrow w_1 + w_2 \geq -2b$. With $w_1 + w_2 < -b$: $-2b \leq w_1 + w_2 < -b \Rightarrow b > 0$, contradicting $b < 0$. No such weights exist.

A-PYQ4. L2 (weight decay): adds $\lambda\|w\|_2^2$ to the loss, shrinking weights smoothly, reducing sensitivity to any single feature (lower variance). **Dropout:** randomly zeroes neurons during training, preventing co-adaptation, approximating an ensemble of thinner networks. **Early stopping:** stop training when validation loss starts rising even as training loss falls, before the model memorizes training-set noise.

A-PYQ5. Input $\rightarrow$ hidden: k neurons $\times$ (n weights + 1 bias) = $k(n + 1)$. Hidden $\rightarrow$ output: m neurons $\times$ (k weights + 1 bias) = $m(k + 1)$.

$$\# \text{params} = k(n + 1) + m(k + 1)$$

A-PYQ6. No. This step function has derivative 0 everywhere except an undefined jump at $x = 0$, so the gradient w.r.t. every upstream weight is 0 almost everywhere. Adam only rescales an existing gradient signal (via momentum and adaptive per-parameter scaling); it cannot manufacture a gradient that doesn't exist. No learning would occur through this activation regardless of optimizer.

A-PYQ7. x-axis: training epochs/iterations. y-axis: loss. Training loss: monotonically decreasing. Validation loss: decreases, reaches a minimum, then increases (overfitting begins). Stop exactly at the validation curve's minimum, even though training loss would keep improving further.

A-PYQ8. For small step $\eta \mathbf{d}$ ($\|\mathbf{d}\| = 1$): $f(w + \eta \mathbf{d}) \approx f(w) + \eta \nabla f(w) \cdot \mathbf{d}$. By Cauchy–Schwarz, $\nabla f \cdot \mathbf{d} \geq -\|\nabla f\|$ with equality at $\mathbf{d} = -\nabla f / \|\nabla f\|$ — the steepest-descent direction. So $f(w - \eta \nabla f) \approx f(w) - \eta \|\nabla f\|^2 < f(w)$, justifying $w \leftarrow w - \eta \nabla f(w)$. **Backprop connection:** backprop computes ∇f (one partial per weight) via the chain rule; the weight update $w \leftarrow w - \eta \partial L / \partial w$ is exactly this gradient-descent step applied per weight.

A-PYQ9. Too-high η : overshoot/divergence/oscillation. Too-low η : painfully slow, can stall on plateaus. AdaGrad's cumulative $G_t = G_{t-1} + g_t^2$ gives frequently/largely-updated parameters a shrunk step and rarely-updated ones a relatively larger step — but G_t only grows, so the effective rate shrinks to 0 over time, halting learning prematurely.

$$\text{AdaGrad: } w_{t+1} = w_t - \frac{\eta}{\sqrt{G_t + \epsilon}} g_t; \quad \text{RMSProp: } E[g^2]_t = \beta E[g^2]_{t-1} + (1-\beta)g_t^2, \quad w_{t+1} = w_t - \frac{\eta}{\sqrt{E[g^2]_t + \epsilon}} g_t$$

RMSProp replaces the ever-growing sum with an exponential moving average, so stale gradients decay away and the effective rate can rise again — it doesn't get permanently killed the way AdaGrad's does.

A-PYQ10. (Using an equivalent self-contained instance, since the original figure is unavailable: $x_1=0.6, x_2=0.1$, target $y=0.3$, $W_1=0.2, W_2=0.4, W_3=0.3(b_1), W_4=0.1, W_5=0.5, W_6=0.3(b_2), W_7=0.3, W_8=0.6, b_1, b_2$ all sigmoid.)

Forward: $h_1 = \sigma(0.46) = 0.6130, h_2 = \sigma(0.41) = 0.6011, \hat{y} = \sigma(0.7446) = 0.6780$.

$$\delta_{\text{out}} = (\hat{y} - y)\hat{y}(1 - \hat{y}) = 0.3780 \times 0.6780 \times 0.3220 = 0.0825.$$

$$\partial L / \partial W_7 = \delta_{\text{out}} h_1 = 0.0506 \Rightarrow W_7^{\text{new}} = 0.3 - 0.1(0.0506) = \boxed{0.2949}$$

$$\partial L / \partial W_8 = \delta_{\text{out}} h_2 = 0.0496 \Rightarrow W_8^{\text{new}} = 0.6 - 0.1(0.0496) = \boxed{0.5950}$$

$$\delta_{h_2} = \delta_{\text{out}} W_8 h_2(1 - h_2) = 0.0825(0.6)(0.2416) = 0.01187$$

$$\partial L / \partial W_4 = \delta_{h_2} x_1 = 0.00713 \Rightarrow W_4^{\text{new}} = 0.1 - 0.1(0.00713) = \boxed{0.0993}$$

A-PYQ11. $\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_j e^{z_j}}$: converts scores into a probability distribution over K classes summing to 1 (raising one lowers the others' share). Sigmoid gives an independent (0, 1) value per unit, no cross-normalization. Use **softmax** for single-label multi-class problems; use **sigmoid** for binary or multi-label classification (multiple classes can be simultaneously true).

A-PYQ12. Adam maintains two exponential moving averages per parameter: $m_t = \beta_1 m_{t-1} + (1 - \beta_1)g_t$ (first moment, i.e. momentum — smooths noisy gradients into a consistent direction) and $v_t = \beta_2 v_{t-1} + (1 - \beta_2)g_t^2$ (second moment, i.e. RMSProp's per-parameter magnitude estimate). Both are bias-corrected ($\hat{m}_t = m_t / (1 - \beta_1^t)$, $\hat{v}_t = v_t / (1 - \beta_2^t)$, correcting the zero-initialization bias which is worst at small t and vanishes as $t \rightarrow \infty$), and combined:

$$w_{t+1} = w_t - \frac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \hat{m}_t$$

Vs. SGD: SGD uses only the raw current gradient with one fixed global η ; Adam uses a smoothed, per-parameter-rescaled direction, adapting to each parameter's own recent gradient statistics. **Advantage of momentum:** damps the noisy, oscillating updates SGD produces in narrow curved valleys, and carries the optimizer through small bumps or saddle regions using accumulated velocity instead of stalling the instant the instantaneous gradient shrinks. Default hyperparameters ($\beta_1=0.9, \beta_2=0.999, \epsilon=10^{-8}$) work well across most problems with little tuning, and because gradients with consistent sign make $\hat{m}_t/\sqrt{\hat{v}_t} \approx \pm 1$, each step is roughly η in size regardless of the raw gradient's scale.

A-PYQ13. Underfitting (high bias): model too simple, high error on both train and test. **Overfitting** (high variance): model too sensitive to training data specifics, low train error but high test error. **L2:** adds $\lambda\|w\|_2^2$, shrinks all weights smoothly toward (not to) 0, reduces variance without eliminating features. **L1:** adds $\lambda\|w\|_1$, drives many weights exactly to 0, giving sparsity / implicit feature selection.

A-PYQ14. Effective (padded) width = $W_1 + 2P$; the filter fits at position 0, then every subsequent position consumes S more of that width:

$$W_2 = \frac{W_1 - F + 2P}{S} + 1, \quad H_2 = \frac{H_1 - F + 2P}{S} + 1$$

Each of K filters has F^2d weights +1 bias: #params = $K(F^2d + 1)$.

A-PYQ15. (Self-contained instance, inputs y_1, y_2 , hidden y_3, y_4 , output y_5 — same numbers as A-PYQ10's forward pass): $y_3 = h_1 = 0.6130$, $y_4 = h_2 = 0.6011$, $y_5 = \hat{y} = 0.6780$. (Method: compute each node's pre-activation as the weighted sum of the previous layer plus bias, then apply sigmoid — repeat layer by layer forward.)

A-PYQ16. *Diagram:* x-axis = model complexity, y-axis = error; Bias² decreases with complexity, Variance increases with complexity, total error is U-shaped (their sum + constant noise floor), minimized at intermediate complexity.

Derivation. $y = f(x) + \varepsilon$, $\mathbb{E}[\varepsilon] = 0$, $\text{Var}(\varepsilon) = \sigma^2$; let $\bar{f} = \mathbb{E}[\hat{f}(x)]$.

$$\mathbb{E}[(y - \hat{f})^2] = \mathbb{E}[(f - \bar{f}) + (\bar{f} - \hat{f}) + \varepsilon]^2$$

Cross terms vanish in expectation ($f - \bar{f}$ constant, $\mathbb{E}[\bar{f} - \hat{f}] = 0$, ε independent mean-zero):

$$= (\bar{f} - f)^2 + \mathbb{E}[(\bar{f} - \hat{f})^2] + \mathbb{E}[\varepsilon^2] = \underbrace{(\bar{f} - f)^2}_{\text{Bias}^2} + \underbrace{\mathbb{E}[(\bar{f} - \hat{f})^2]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{noise}}$$

A-PYQ17. $F = 1$: $W_2 = (n - 1)/1 + 1 = n$, output same size as input; each pixel is a linear recombination across channels only, zero spatial mixing. $F = n$: $W_2 = (n - n)/1 + 1 = 1$, output collapses to a single value per filter — effectively a fully-connected layer over the whole image, discarding all spatial structure.

A-PYQ18. Setup: $x_1, x_2 \rightarrow h$ (sigmoid, weights w_1, w_2) $\rightarrow \hat{y}$ (sigmoid, weight w_3), $L = \frac{1}{2}(y - \hat{y})^2$. Let $\delta_y = (\hat{y} - y)\hat{y}(1 - \hat{y})$.

$$\text{grad}(w_3) = \delta_y \cdot h; \quad \delta_h = \delta_y w_3 h(1 - h)$$

$$\text{grad}(w_1) = \delta_h \cdot x_1 = \delta_y w_3 h(1 - h) x_1; \quad \text{grad}(w_2) = \delta_y w_3 h(1 - h) x_2$$

A-PYQ19. SGD: update from a single example's gradient; noisy, cheap per step. **Momentum GD:** accumulates an exponentially-weighted velocity from past gradients, $v_t = \beta v_{t-1} - \eta \nabla f(w_t)$, smoothing the noise and accelerating in a consistent direction. **NAG:** like Momentum, but the gradient is evaluated at the look-ahead point $w_t + \beta v_{t-1}$, letting it "peek ahead" and decelerate before overshooting a minimum.

A-PYQ20. Three excitatory inputs, weight 1 each, threshold $\theta = 3$: $y = 1$ iff $x_1 + x_2 + x_3 \geq 3$ — true only when all three are 1.

A-PYQ21. (a) **False** — simple models have high bias but *low* variance. (b) **False** — desirable is low bias *and* low variance. (c) **False** — Error = Bias²+Variance+noise; both contribute. (d) **False** — in NAG the momentum term is added first (to get the look-ahead point), then the gradient (computed at that look-ahead point) is added.

A-PYQ22. $m_t = (1 - \beta_1) \sum_{i=1}^t \beta_1^{t-i} g_i$; if $g_i = g$ constant, $m_t = g(1 - \beta_1^t)$ — biased low by factor $(1 - \beta_1^t)$, worst at small t (since $m_0 = 0$). Dividing by exactly this factor, $\hat{m}_t = m_t / (1 - \beta_1^t)$, removes the bias. As $t \rightarrow \infty$, $\beta_1^t \rightarrow 0$ so the correction factor $\rightarrow 1$ and the correction becomes unnecessary — it fades out on its own. Same argument applies to v_t, β_2 .

A-PYQ23. Without a non-linearity, stacked linear layers collapse into one linear layer, so activations are needed for the network to represent non-linear functions at all. A good activation is differentiable a.e., non-saturating over most of its range, ideally zero-centered, and cheap to compute. Sigmoid: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (derived from $\sigma(z) = (1 + e^{-z})^{-1}$ via the chain rule). **Merit:** smooth, bounded, interpretable as a probability. **Drawbacks:** saturates for large $|z|$ (vanishing gradient), not zero-centered, costly exponential.

A-PYQ24. (i) Sigmoid+MSE's gradient has factor $\hat{y}(1 - \hat{y}) \rightarrow 0$ when confidently wrong — learning stalls exactly when correction is needed most; MSE's classification loss surface is also non-convex. (ii) $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. (iii) $\hat{y} = 0.9, y = 0$: $L = -\log(0.1) = \boxed{2.303}$. (iv) $y=1, \hat{y}=0.1$: $-\log(0.1)=2.303$; $y=0, \hat{y}=0.2$: $-\log(0.8)=0.223$; $y=0, \hat{y}=0.7$: $-\log(0.3)=1.204$. $J = (2.303 + 0.223 + 1.204)/3 = \boxed{1.243}$.

A-PYQ25. Using $W_2 = (W_1 - F)/S + 1$ ($S=1, P=0$) and halving per pool:

Layer	Output	Params
INPUT	$128 \times 128 \times 3$	—
CONV-9-32	$120 \times 120 \times 32$	$32(9^2 \cdot 3 + 1) = 7808$
POOL-2	$60 \times 60 \times 32$	0
CONV-5-64	$56 \times 56 \times 64$	$64(5^2 \cdot 32 + 1) = 51264$
POOL-2	$28 \times 28 \times 64$	0
CONV-5-64	$24 \times 24 \times 64$	$64(5^2 \cdot 64 + 1) = 102464$
POOL-2	$12 \times 12 \times 64$	0
FC-10	$9216 \rightarrow 10$	92170
FC-3	$10 \rightarrow 3$	33

Total = 253,739.

A-PYQ26. 3 hidden units. Let $s = x_1 + x_2 + x_3$; parity = 1 iff s odd. Build $h_1 = \mathbb{K}[s \geq 1]$, $h_2 = \mathbb{K}[s \geq 2]$, $h_3 = \mathbb{K}[s \geq 3]$ (threshold units, weights 1, 1, 1, thresholds 0.5, 1.5, 2.5). Output = $\text{sign}(h_1 - h_2 + h_3 - 0.5)$: at $s=0:0$; $s=1:1$; $s=2:1-1+0=0$; $s=3:1-1+1=1$ — correct parity using only 3 hidden units.

A-PYQ27. (Method demonstrated on the fresh instance in M2.6's answer below — apply identically to whatever image/filter is given: slide the filter, dot product each position, then max-pool, then $P = (F - 1)/2$ for same-size padding with stride 1.)

A-PYQ28. L1, L2 (weight decay), dropout, early stopping, dataset augmentation, batch normalization (regularizing side-effect), noise injection (inputs/weights), parameter sharing/tying, ensembling/bagging — any 8.

A-PYQ29. Binary classification: **Binary Cross-Entropy**, $L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$. (Multi-class: categorical cross-entropy, $L = -\sum_k y_k \log \hat{y}_k$.)

5.2 Additional Questions Answers

A-O1. 2-input NOR = 1 only when both inputs are 0. Use *inhibitory* inputs: both x_1, x_2 inhibitory, threshold $\theta = 0$: $y = 1$ iff no inhibitory input active and $0 \geq \theta$, i.e. iff both $x_1 = x_2 = 0$. This correctly gives NOR.

A-O2. By the convergence bound, mistakes $\leq R^2/\gamma^2 = 5^2/0.5^2 = 25/0.25 = \boxed{100}$.

A-O3. $t=1$: $m_1=0.1(1.2)=0.12$, $v_1=0.001(1.44)=0.00144$; $\hat{m}_1=0.12/0.1=1.2$, $\hat{v}_1=0.00144/0.001=1.44$, $\sqrt{\hat{v}_1}=1.2$. $w_1 = 2.0 - 0.01(1.2/1.2) = 2.0 - 0.01 = \boxed{1.99}$.

$t=2$: $m_2=0.9(0.12)+0.1(-0.5)=0.058$, $v_2=0.999(0.00144)+0.001(0.25)=0.001689$; $\hat{m}_2=0.058/0.19=0.3053$, $\hat{v}_2=0.001689/0.001999=0.8447$, $\sqrt{\hat{v}_2}=0.9190$. $\Delta w = 0.01(0.3053/0.9190) = 0.003322$. $w_2 = 1.99 - 0.003322 = \boxed{1.9867}$.

Note m_2 stays positive despite $g_2 < 0$: momentum's accumulated history from g_1 still dominates.

A-O4. $\nabla f(w_0) = 2(1.0) = 2.0$. **(a) Momentum:** $v = \beta v_{\text{prev}} - \eta \nabla f(w_0) = 0.9(0.3) - 0.1(2.0) = 0.27 - 0.2 = 0.07$; $w_1 = 1.0 + 0.07 = \boxed{1.07}$. **(b) NAG:** look-ahead $= w_0 + \beta v_{\text{prev}} = 1.0 + 0.27 = 1.27$; gradient there $= 2(1.27) = 2.54$; $v = 0.27 - 0.1(2.54) = 0.27 - 0.254 = 0.016$; $w_1 = 1.0 + 0.016 = \boxed{1.016}$. NAG takes a much smaller step here: evaluating the gradient at the look-ahead point (which has already moved further from the minimum) gives a larger gradient magnitude, which decelerates the update — exactly the “look before you leap” correction NAG is designed for.

A-O5. $\text{Var}(y) = \text{Var}(\sum_i w_i x_i) = \sum_i \text{Var}(w_i x_i) = \sum_i \text{Var}(w_i) \text{Var}(x_i)$ (independence, zero mean) $= n \text{Var}(w) \text{Var}(x)$ (identical variance per input). Setting $\text{Var}(y) = \text{Var}(x)$: $n \text{Var}(w) = 1 \Rightarrow \boxed{\text{Var}(w) = 1/n}$. (The full Xavier/Glorot formula $2/(n_{\text{in}} + n_{\text{out}})$ balances this forward-pass argument with an identical backward-pass argument giving $1/n_{\text{out}}$, averaging the two.)

A-O6. ReLU zeroes roughly half its inputs (all negative pre-activations), so on average only $\sim$ half the units pass signal forward. Using Xavier’s $1/n_{\text{in}}$ (calibrated assuming symmetric signal preservation) would let the forward variance shrink by $\sim$ half at every layer, vanishing in deep networks. He’s $2/n_{\text{in}}$ doubles the variance to exactly compensate for that expected halving, keeping signal variance stable layer to layer.

A-O7. Let $m \sim \text{Bernoulli}(p)$ be the keep-mask, h the pre-dropout activation. Inverted dropout output: $h_{\text{drop}} = \frac{m \cdot h}{p}$. Since h is fixed given the input (randomness is only in m):

$$\mathbb{E}[h_{\text{drop}}] = \frac{\mathbb{E}[m] \cdot h}{p} = \frac{p \cdot h}{p} = h$$

The expected output during training already equals the true value h , so at test time the network can be run as-is with no dropout and no rescaling — this is exactly why it’s called *inverted* dropout (the scaling is moved to train time), unlike vanilla dropout which requires multiplying by p at test time instead.

A-O8. $z_i = \gamma \hat{x}_i + \beta$ for each of the m batch elements; γ, β are shared (broadcast) across the batch, so their total gradient sums the contribution from every i :

$$\frac{\partial L}{\partial \gamma} = \sum_{i=1}^m \frac{\partial L}{\partial z_i} \frac{\partial z_i}{\partial \gamma} = \sum_{i=1}^m \delta_i \hat{x}_i, \quad \frac{\partial L}{\partial \beta} = \sum_{i=1}^m \delta_i \frac{\partial z_i}{\partial \beta} = \sum_{i=1}^m \delta_i$$

A-O9. Layer 1: $W_2 = (32 - 5 + 2(2))/1 + 1 = (32 - 5 + 4) + 1 = 32$; output $32 \times 32 \times 16$. Params $= 16(5^2(3) + 1) = 16(76) = \boxed{1216}$. **Layer 2:** input $32 \times 32 \times 16$; $W_2 = (32 - 4 + 0)/2 + 1 = 28/2 + 1 = \boxed{15}$; output $15 \times 15 \times 32$. Params $= 32(4^2(16) + 1) = 32(257) = \boxed{8224}$.

A-O10. Forward: $a_h = 0.6(0.5) + 0.3(-0.2) + 0.1 = 0.34$; $h = \tanh(0.34) = 0.3275$. $a_y = 0.8(0.3275) - 0.2 = 0.062$; $\hat{y} = \sigma(0.062) = 0.5155$.

$$\delta_{\text{out}} = (\hat{y} - y)\hat{y}(1 - \hat{y}) = (0.1155)(0.2498) = 0.02885.$$

$$\partial L / \partial w_3 = \delta_{\text{out}} h = 0.02885(0.3275) = 0.009448 \Rightarrow w_3^{\text{new}} = 0.8 - 0.15(0.009448) = \boxed{0.79858}$$

$$\delta_h = \delta_{\text{out}} w_3 (1 - h^2) = 0.02885(0.8)(1 - 0.1073) = 0.02885(0.8)(0.8927) = 0.020604$$

$$\partial L / \partial w_1 = \delta_h x_1 = 0.020604(0.5) = 0.010302 \Rightarrow w_1^{\text{new}} = 0.6 - 0.15(0.010302) = \boxed{0.59846}$$

(Note the tanh derivative used: $\varphi'(a) = 1 - \tanh^2(a) = 1 - h^2$, in place of sigmoid's $h(1 - h)$ — same recursion, different local slope, exactly as generalized in Section 6.)

5.3 Mock Paper 1 Answers

M1.1 = A-PYQ1. M1.2 = A-PYQ2 (4-mark version). M1.4 = A-PYQ19. M1.5 = A-PYQ9. M1.6 = A-PYQ14. M1.8 = A-PYQ22. Fresh solutions for M1.3 and M1.7 below.

A-M1.3. Forward: $h_1 = \sigma(0.25(0.4)+0.35(0.3)+0.1) = \sigma(0.305) = 0.5757$; $h_2 = \sigma(0.4(0.4)+0.2(0.3)+0.15) = \sigma(0.37) = 0.5915$; $\hat{y} = \sigma(0.5(0.5757)+0.45(0.5915)+0.1) = \sigma(0.6540) = 0.6580$.

$$\delta_{\text{out}} = (0.6580 - 0.25)(0.6580)(0.3420) = 0.09182.$$

$$\partial L / \partial W_7 = \delta_{\text{out}} h_1 = 0.09182(0.5757) = 0.05285 \Rightarrow W_7^{\text{new}} = 0.5 - 0.2(0.05285) = \boxed{0.48943}$$

$$\delta_{h_1} = \delta_{\text{out}} W_7 h_1 (1 - h_1) = 0.09182(0.5)(0.2443) = 0.011216 \quad \partial L / \partial W_2 = \delta_{h_1} x_2 = 0.011216(0.3) = 0.003365 \Rightarrow W_2^{\text{new}} = 0.35 - 0.2(0.003365) = \boxed{0.34933}$$

$$\delta_{h_2} = \delta_{\text{out}} W_8 h_2 (1 - h_2) = 0.09182(0.45)(0.2416) = 0.009982 \quad \partial L / \partial W_6 = \delta_{h_2}(1) = 0.009982 \Rightarrow W_6^{\text{new}} = 0.15 - 0.2(0.009982) = \boxed{0.14800}$$

A-M1.7. (i)/(ii) Same reasoning/formula as A-PYQ24(i)-(ii). (iii) $y=1, \hat{y}=0.8$: $L = -\log(0.8) = \boxed{0.2231}$. (iv) $y=1, \hat{y}=0.7$: $-\log(0.7)=0.3567$; $y=1, \hat{y}=0.4$: $-\log(0.4)=0.9163$; $y=0, \hat{y}=0.3$: $-\log(0.7)=0.3567$; $y=0, \hat{y}=0.6$: $-\log(0.4)=0.9163$. $J = (0.3567+0.9163+0.3567+0.9163)/4 = \boxed{0.6365}$.

5.4 Mock Paper 2 Answers

M2.1 = A-PYQ3. M2.2 = A-PYQ16. M2.3 = A-PYQ13. M2.4 = A-PYQ11. M2.5 = A-PYQ12. M2.7 = A-PYQ28. Fresh solutions for M2.6 and M2.8 below.

A-M2.6. Sliding the Laplacian-style filter over the 5×5 input (stride 1, no padding, 3×3 output). Each output = top+bottom+left+right−4×center of the covered window. Computing all 9 positions:

$$\text{Conv output} = \begin{pmatrix} -6 & 7 & 1 \\ 2 & -11 & 7 \\ -1 & 8 & -6 \end{pmatrix}$$

2×2 max pooling, stride 1 (output 2×2): top-left $\max(-6, 7, 2, -11) = 7$; top-right $\max(7, 1, -11, 7) = 7$; bottom-left $\max(2, -11, -1, 8) = 8$; bottom-right $\max(-11, 7, 8, -6) = 8$.

$$\text{Max-pooled output} = \begin{pmatrix} 7 & 7 \\ 8 & 8 \end{pmatrix}$$

Padding for same-size convolution output (5×5): $P = (F - 1)/2 = (3 - 1)/2 = \boxed{1}$.

A-M2.8. #params = $k(n + 1) + m(k + 1) = 8(7) + 4(9) = 56 + 36 = \boxed{92}$.